

장진성

UnrealEngine4 포트폴리오

프로젝트 개요

- 목표
 - 원즈를 참고하여 3D로 구현
 - UMG를 이용한 UI구현 및 상호작용
 - 턴제게임 구조 설계 및 구현
 - 한 턴에서의 카메라 전환 설계 및 구현
 - 로프시스템 구현
-
- 데이터테이블을 이용한 아이템 추가

- 무기

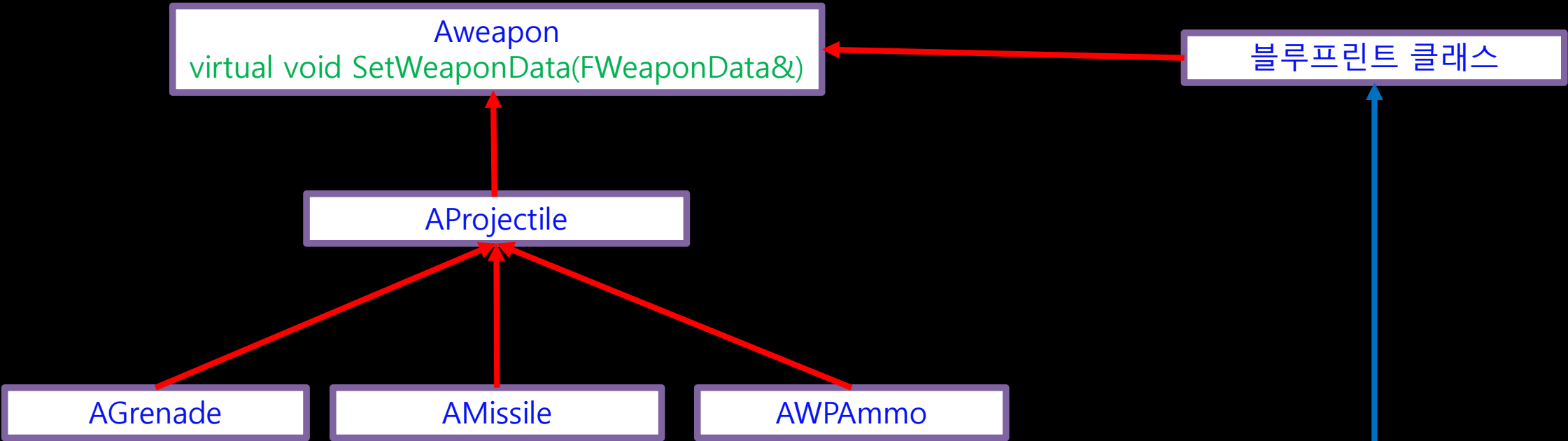
빨간색 화살표는 상속입니다.

```
USTRUCT()
struct FWeaponData : public FTableRowBase
{
    GENERATED_BODY()

    FWeaponData() : AttackPower(10.0f){}
}

USTRUCT()
struct FProjectileData : public FWeaponData
{
    GENERATED_BODY()

    FProjectileData() : Radius(5.0f),
        MaxSpeed(3000.0f), InitialSpeed(1000.f),
        Weight(10.0f), ExplodeRange(100.0f)
    {}
}
```



데이터 테이블

데이터 테이블 디테일

검색

	행 이름	Radius	Max Speed	Initial Speed	Weight	Explode Range	Weapon Name	Attack Power	Weapon Class
1	Grenade	5.000000	7000.000000	1000.000000	10.000000	300.000000	Grenade	10.000000	BlueprintGeneratedClass'/Game/Blueprints/Grenade.Grenade_C'
2	Rocket	10.083142	7000.000000	1637.048096	17.328495	480.921783	Rocket	50.000000	BlueprintGeneratedClass'/Game/Blueprints/RocketLauncher.RocketLaur
3	PowerRocket	100.000000	10000.000000	2000.000000	20.000000	1000.000000	PowerRocket	100.000000	BlueprintGeneratedClass'/Game/Blueprints/PowerRocket.PowerRocket_

- Aweapon 클래스를 기본으로 상속받아 수류탄, 미사일 등 다른 기능을 가진 다양한 클래스로 확장합니다.
- 클래스를 상속받아 블루프린트 클래스를 만들고 메쉬를 설정합니다.
- 데이터테이블에서 무기의 세부설정과 클래스를 관리합니다.
- 인게임에서 데이터테이블을 참조해 Weapon 액터를 생성합니다.

- 무기

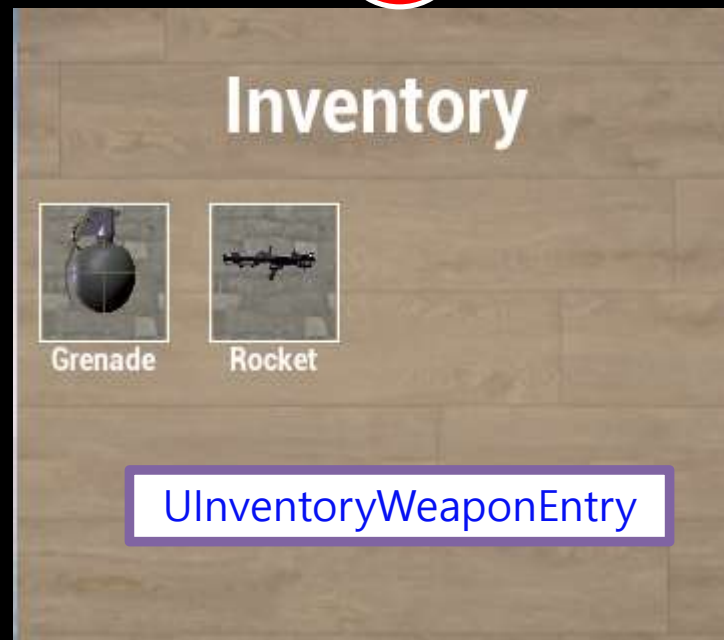
데이터 테이블									
검색									
	행 이름	Radius	Max Speed	Initial Speed	Weight	Explode Range	Weapon Name	Attack Power	Weapon Class
1	Grenade	5.000000	7000.000000	1000.000000	10.000000	300.000000	Grenade	10.000000	BlueprintGeneratedClass'/Game/Blueprints/Grenade.Grenade_C'
2	Rocket	10.083142	7000.000000	1637.048096	17.328495	480.921783	Rocket	50.000000	BlueprintGeneratedClass'/Game/Blueprints/RocketLauncher.RocketLaur
3	PowerRocket	100.000000	10000.000000	2000.000000	20.000000	1000.000000	PowerRocket	100.000000	BlueprintGeneratedClass'/Game/Blueprints/PowerRocket.PowerRocket_C'

1



AltemActor
UMeshComponent* Mesh
UNiagaraComponent* Effect

2



3



AWeapon

- 데이터테이블에 저장된 WeaponClass는 3개의 클래스에서 사용됩니다.
- 1. AltemActor – 캐릭터가 습득할 수 있는 아이템 클래스입니다. 위젯과 이펙트를 가지고 있으며 캐릭터와 Overlap 됐을 시 'E'키를 눌러 습득하는 기능이 있습니다. WeaponClass를 이용해 Mesh를 초기화합니다.
- 2. UInventoryWeaponEntry – 텍스처에 액터를 렌더링하고, UTileView의 Entry에 사용합니다.
- 3. Aweapon – WeaponClass는 기본적으로 Aweapon 클래스입니다. 간단하게 그대로 스폰시켜 플레이어가 무기로 사용합니다.

- 무기 위젯엔트리

```
void UInventoryWeaponWidget::AddListItem(const FWPIItem& Item)
{
    UE_LOG(LogTemp, Warning, TEXT("AddListItem"));

    if (PlayerInventory.IsValid())
    {
        auto ItemName = Item.ItemName;
        UMaterialInstanceDynamic* ItemMaterial;

        //해당 아이템으로 텍스처, 메테리얼이 생성되었는지 확인
        auto pItemMaterial:UMaterialInstanceDynamic** = WeaponListItems.Find(ItemName);
        if (pItemMaterial == nullptr)
        {
            //아이템을 렌더링할 텍스처 생성
            1 UTextureRenderTarget2D* ItemTexture =
              NewObject<UTextureRenderTarget2D>(Outer, this);
            2 ItemRenderToTexture(Item, ItemTexture);

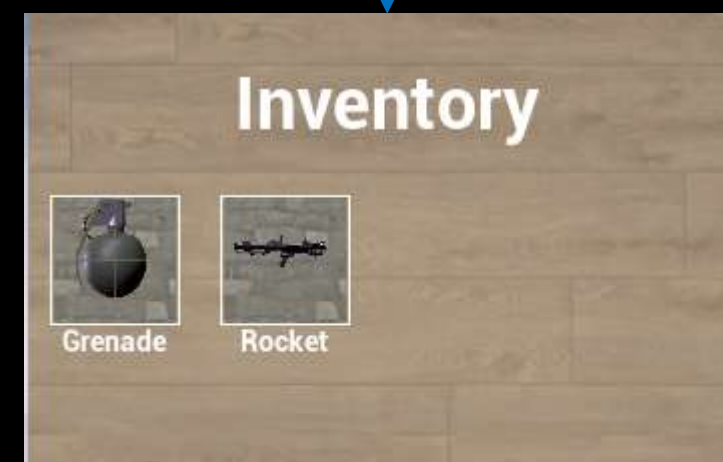
            //메테리얼 텍스처 파라미터에 렌더링한 텍스처 설정
            UMaterialInstanceDynamic* MaterialInstance = UMaterialInstanceDynamic::Create(
                MaterialInstance->SetTextureParameterValue(TEXT("ItemTexture"), ItemTexture);
            ItemMaterial = MaterialInstance;
        }
        else
            ItemMaterial = *pItemMaterial;

        3 auto ListItem = NewObject<UListItem>(Outer, this, ItemName);
        ListItem->Init(Item, ItemMaterial);

        ItemInventory->AddItem(ListItem);
    }
}
```

UInventoryWeaponWidget
UTileView* ItemInventory
Void AddListItem(const FWPIItem& Item)

UCaptureHelper
void DrawActorToTexture
(UClass* ActorClass, UTextureRenderTarget2D* Target,
UInt32 SizeX, UInt32 SizeY)



4 UInventoryWeaponEntry
Virtual void NativeOnListItemObjectSet
(UObject* ListItemObject)
Uimage* WeaponImage

1. 인벤토리 위젯 클래스에서 텍스처를 생성합니다.
2. UCaptureHelper 클래스를 이용해 텍스처에 무기(아이템)을 렌더링합니다.
3. 텍스처로 Item을 초기화하고 TileView에 추가합니다.
4. Entry는 추가된 Item의 텍스처를 이용해 Uimage의 이미지를 초기화합니다.

- 텍스처 렌더링1

1. 원하는 크기로 텍스처를 초기화합니다.
2. 텍스처 렌더링을 위해 잠깐 사용할 월드를 생성합니다.
3. 현재 월드에 있는 평행광을 복사해 임시월드에서 추가합니다.
4. UClass를 사용해 액터를 생성합니다.
Uclass는 일반적으로 데이터테이블에서 얻어옵니다.
5. 캡처 컴포넌트를 생성하고 액터에 부착합니다.

```
void UCaptureHelper::DrawActorToTexture(UClass* ActorClass, UTextureRenderTarget2D* RenderTarget,
uint32 sizeX, uint32 sizeY)
{
    check(RenderTarget && ActorClass);

    1 //렌더타겟 초기화
    RenderTarget->InitAutoFormat(sizeX, sizeY);
    RenderTarget->ClearColor = FLinearColor(InR:0.f, InG:1.f, InB:0.f, InA:1.f);
    RenderTarget->UpdateResource();

    2 //캡처할 액터를 스폰시킬 임시 월드 생성
    UWorld* OffscreenWorld = UWorld::CreateWorld(EWorldType::GamePreview, bInformEngineOfWorld: true);

    if(World == nullptr)
    {
        World = GetWorld();
        check(World);
    }

    3 //임시 월드에 현재 월드와 같은 평행광 추가
    for (TActorIterator<ADirectionalLight> It(World); It; ++It)
    {
        auto Light = *It;

        FActorSpawnParameters Params;
        Params.Template = Light;
        auto SpawnedLight = OffscreenWorld->SpawnActor<ADirectionalLight>(Params);
    }

    //렌더링할 액터를 임시월드에서 생성
    4 const FTransform ActorTr(InRotation: FRotator(InPitch:0.0f, InYaw:0.0f, InRoll:0.0f),
        InTranslation: FVector(InX:0.0f, InY:0.0f, InZ:0.0f));
    AActor* SpawnedActor = OffscreenWorld->SpawnActor<AActor>(ActorClass, ActorTr);

    5 //스폰한 액터를 촬영할 캡처컴포넌트 생성
    USceneCaptureComponent2D* SceneCaptureComponent = NewObject<USceneCaptureComponent2D>(SpawnedActor);

    SceneCaptureComponent->AttachToComponent(InParent: SpawnedActor->GetRootComponent(),
        FAttachmentTransformRules::KeepRelativeTransform);
}
```

- 텍스처 렌더링2

6. 액터의 바운딩박스를 가져옵니다.

7. 바운딩박스에서 가장 긴 축 2개를 찾습니다.

char Flag 변수의 하위3비트를
각 축에 해당하는 비트로 사용해
2비트를 1로 체크합니다.



- 위 사진에서 가장 긴 축은 X축과 Z축 입니다.
- Flag는 0000 0101이 됩니다.

6

```
//렌더링 할 액터의 바운딩박스를 가져옵니다.  
FVector BoxExtent;  
AWeapon* SpawnedWeapon = Cast<AWeapon>(SpawnedActor);  
UMeshComponent* WeaponMeshComponent = nullptr;  
  
if(SpawnedWeapon)  
{  
    WeaponMeshComponent = SpawnedWeapon->GetMesh();  
}  
  
if(WeaponMeshComponent)  
{  
    BoxExtent = SpawnedWeapon->GetMesh()->Bounds.BoxExtent;  
}
```

//가장 큰 면의 길이

```
float MaxExtent = BoxExtent.GetAbsMax();
```

//0000 0ZYX

```
char Flag = 0;
```

7

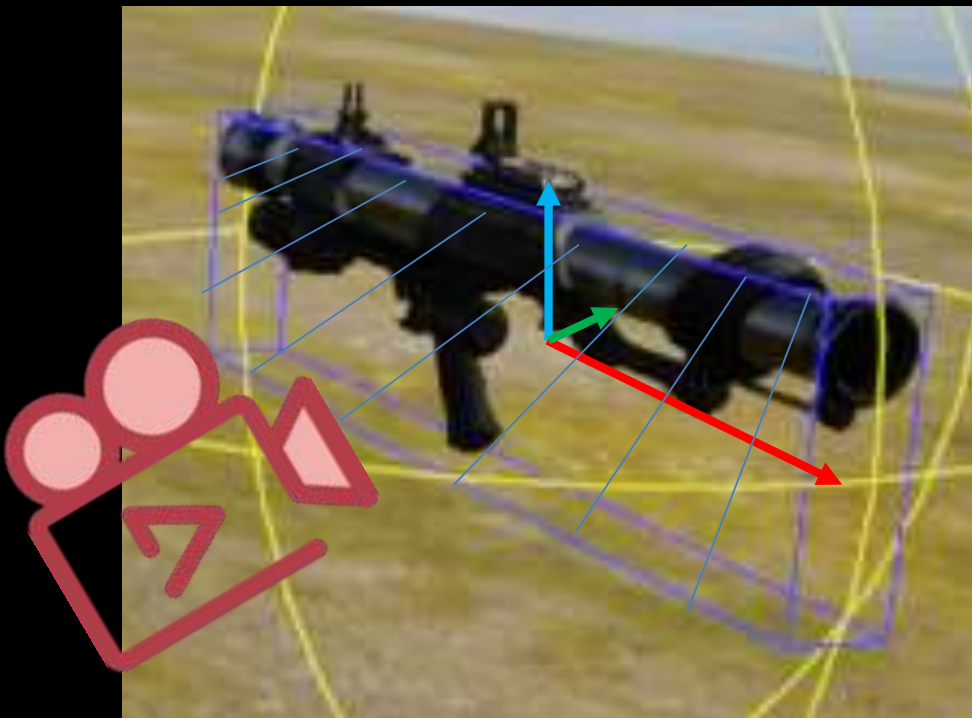
//바운딩박스의 xyz축 중에서, 가장 긴 두 축을 찾습니다.

```
if (BoxExtent.X >= BoxExtent.Y)  
{  
    Flag += 1;  
    if (BoxExtent.Y >= BoxExtent.Z)  
        Flag += 2;  
    else  
        Flag += 4;  
}  
else  
{  
    Flag += 2;  
    if (BoxExtent.X >= BoxExtent.Z)  
        Flag += 1;  
    else  
        Flag += 4;  
}
```


- 텍스처 렌더링3

8. 캡처 카메라가 바운딩박스의 가장 넓은 면을 바라보도록 캡처 컴포넌트의 위치와 회전을 설정합니다. 가장 길이가 긴 축 2개로 이루어진 평면을 바라봅니다.

8



- 위 사진의 Flag가 0000 0101이기때문에 XZ평면을 바라보도록 캡처 컴포넌트를 설정합니다.
- 이제 어떤 메쉬든 가장 넓은 면을 직교투영으로 텍스처에 렌더링 할 수 있습니다.

```
FRotator CameraRot;
FVector CameraMoveDir;

//Flag AND 111
//가장 넓은 면을 바라보도록 카메라를 설정
switch(Flag & 7)
{
case 3: //011 XY Axis, Look XY Plane.
    CameraMoveDir.Set(InX: 0.f, InY: 0.f, InZ: 1.f);
    CameraRot.Pitch = -90.f;
    break;
case 5: //101 XZ Axis, Look XZ Plane
    CameraMoveDir.Set(InX: 0.f, InY: 1.f, InZ: 0.f);
    CameraRot.Yaw = 90.f;
    break;
case 6: //110 YZ Axis, Look YZ Plane
    CameraMoveDir.Set(InX: 1.f, InY: 0.f, InZ: 0.f);
    break;
}

//SceneCapture 카메라를 직교투영으로 설정
ViewInfo.ProjectionMode = ECameraProjectionMode::Orthographic;
ViewInfo.OrthoWidth = MaxExtent * 2.5f;
ViewInfo.OrthoNearClipPlane = 0.0f;
ViewInfo.OrthoFarClipPlane = MaxExtent*3;

SceneCaptureComponent->SetCameraView(ViewInfo);
SceneCaptureComponent->CaptureScene();

//사용한 임시월드 파괴
OffscreenWorld->DestroyWorld(bInformEngineOfWorld: true);
```


- 무기 사용

- 플레이어의 무기 사용은
좌클릭 Press(무기 생성) ->
클릭 중(궤도 계산) ->
좌클릭 Release(무기 사용)
세 단계를 거쳐서 실행됩니다.

1. 장착아이템 클래스에서 현재 장착중인 무기의 데이터를 가져옵니다. 데이터테이블에서 얻어온 무기의 정보와 UClass가 미리 저장돼 있습니다.
2. SpawnActorDeferred로 무기액터를 생성하여, 컴포넌트의 초기화를 막습니다.
3. WeaponData를 설정합니다. SetWeaponData함수는 AWeapon 클래스의 가상함수입니다. 예시로 Aprojectile 클래스는 WeaponData를 받아 ProjectileMovementComponent의 무게, 최대속도 등을 설정합니다.

1

```
//현재 장착중인 무기 데이터를 가져옵니다.
const auto WeaponData = const pair<FWPItem, unique_ptr<FWeaponData>>* = mEquipments->GetWeapon();

//장착중인 무기가 없으면 리턴
if (WeaponData == nullptr)
{
    return;
}

if (mCurrentWeapon != nullptr)
{
    mCurrentWeapon->Destroy();
    mCurrentWeapon = nullptr;
}

TSubclassOf<AWeapon> WeaponUClass = WeaponData->second->WeaponClass;

if (WeaponUClass != nullptr)
{
    UWorld* const World = GetWorld();
    if (World != nullptr)
    {
        FActorSpawnParameters ActorSpawnParams;
        ActorSpawnParams.SpawnCollisionHandlingOverride =
            ESpawnActorCollisionHandlingMethod::AdjustIfPossibleButDontSpawnIfColliding;

        //일반 Spawn을 사용하면 ProjectileMovementComponent때문에 무기가 바로 발사됨.
        mCurrentWeapon = World->SpawnActorDeferred<AWeapon>
            (WeaponUClass, FTransform(), Owner: this, Instigator: this);

        //무기 데이터 설정
        mCurrentWeapon->SetWeaponData(WeaponData->second.get());
        mCurrentWeapon->SetActorEnableCollision(false);

        //무기작용 소켓에 부착
        mCurrentWeapon->AttachToComponent(mWeaponPos, FAttachmentTransformRules::SnapToTargetIncludingScale);
    }
}
```

2

```
//일반 Spawn을 사용하면 ProjectileMovementComponent때문에 무기가 바로 발사됨.
mCurrentWeapon = World->SpawnActorDeferred<AWeapon>
(WeaponUClass, FTransform(), Owner: this, Instigator: this);
```

3

```
//무기 데이터 설정
mCurrentWeapon->SetWeaponData(WeaponData->second.get());
mCurrentWeapon->SetActorEnableCollision(false);
```

```
//무기작용 소켓에 부착
```

```
mCurrentWeapon->AttachToComponent(mWeaponPos, FAttachmentTransformRules::SnapToTargetIncludingScale);
```

- 무기 사용2

- 좌클릭을 누르고 있을 때 실행되는 함수입니다.
- 발사체의 궤적을 렌더링합니다.

1. 엔진함수로 투사체의 궤적위치를 계산해 Result 배열에 저장합니다.
2. 궤적을 나타내는 액터, 아래 사진에서 [노란색 구 액터]를 원하는 개수로 스폰합니다.
3. 타격지점에 폭발범위 만큼의 크기를 가진 이펙트를 스폰합니다.
이 이펙트는 데칼과 구체에 메테리얼을 적용해 블루프린트로 제작하였습니다.



```
//투사체 궤적 계산
1 UGameplayStatics::PredictProjectilePath(GetWorld(), Params, [ &] Result);

FVector ImpactPoint = Result.HitResult.ImpactPoint;

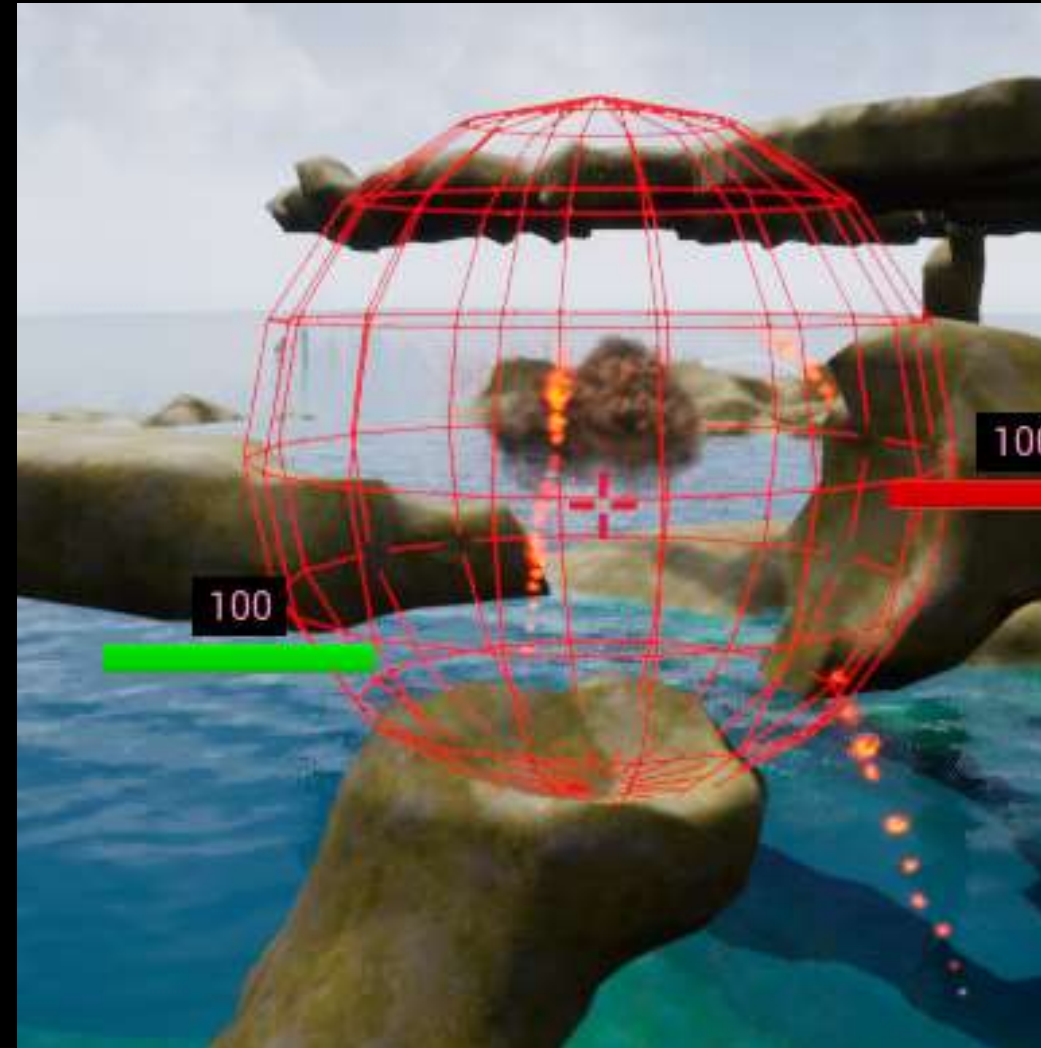
//궤적을 나타내는 액터를 TrajectoryNum 개수만큼 스폰
2 static float TrajectoryNum = 15;
auto& posArr : TArray<FPredictProjectilePathPointData>& = Result.PathData;
float stride = posArr.Num() / TrajectoryNum;

UWorld* World = GetWorld();
if(posArr.Num() > 0)
{
    for (float i = 1; i <= posArr.Num() - 1; i += stride)
    {
        int idx = FMath::RoundToInt(i);
        auto pos : FPredictProjectilePathPointData = posArr[idx];
        World->SpawnActor(mTrajectoryActor, &pos.Location);
    }
}

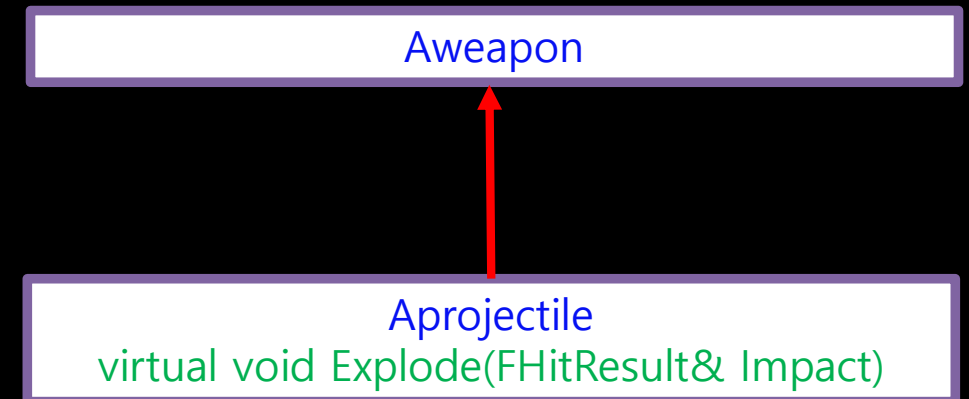
//타격지점에 폭발범위만큼 원형이펙트 스폰
3 if(Result.HitResult.bBlockingHit)
{
    AActor* FireRangeActor = World->SpawnActor(mFireRangeActor, &ImpactPoint);
    FVector Origin, Extents;
    FireRangeActor->GetActorBounds(bOnlyCollidingComponents: false, [ &] Origin, [ &] BoxExtent);
    float Scale = mProjectileInfo.ExplodeRange / Extents.X;

    FireRangeActor->SetActorScale3D(FVector(InX: Scale, InY: Scale, InZ: Scale));
}
```


- Voxel Plugin



- 폭발에 의한 지형 파괴를 구현하기 위해 Voxel Plugin을 사용했습니다.
- Aprojectile 클래스의 Explode 함수에서 플러그인의 함수를 호출해 충돌지점에서 폭발범위만큼 지형을 파괴합니다.



- 로프 이동 1

```
void AMyRope::GetTangentVector(FVector& RightTangentVec, FVector& UpTangentVec)
{
    if (!OwnerCharacter.IsValid())
        return;

    //로프를 소유한 캐릭터의 위치
    APlayerCharacter* Character = OwnerCharacter.Get();
    FVector PlayerLocation = Character->GetActorLocation();

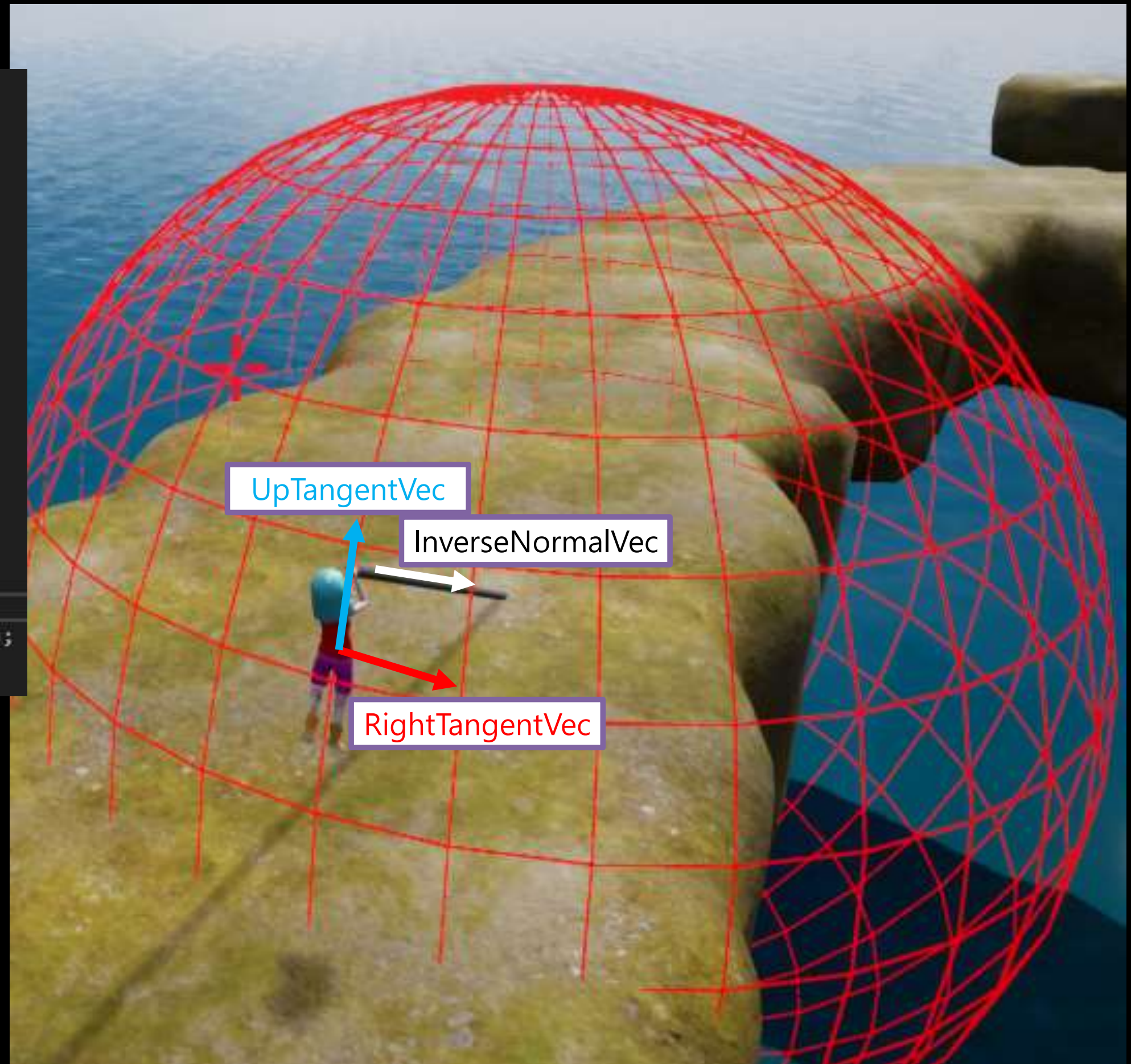
    //로프의 시작점 위치
    FVector SphereMidPoint = GetActorLocation();

    //캐릭터에서 로프로 향하는 벡터
    FVector InverseNormalVec = SphereMidPoint - PlayerLocation;

    //캐릭터의 오른쪽방향을 나타내는 접벡터를 구함
    RightTangentVec = -FVector::CrossProduct(
        A: InverseNormalVec.GetSafeNormal(), B: FVector::UpVector);

    //캐릭터의 위쪽 방향을 나타내는 접벡터를 구함
    UpTangentVec = -FVector::CrossProduct(A: RightTangentVec, B: InverseNormalVec);
}
```

- 로프를 사용할 때는 기존과 다른 움직임을 적용합니다.
- 로프가 연결된 지점과 캐릭터 사이의 거리를 반지름으로 사용하는 구를 가정합니다.
- 구의 표면에 있는 캐릭터위치에서 우측방향 접벡터와 위쪽방향 접벡터를 구해서 이동에 사용합니다.



- 로프 이동 2

```
void AMyRope::AddForceCharacter(float XInput, float YInput)
{
    if (!OwnerCharacter.IsValid())
        return;

    APlayerCharacter* Character = OwnerCharacter.Get();
    UCharacterMovementComponent* MovementComponent = Character->GetCharacterMovement();

    //현재 위치에서 탄젠트벡터 계산
    FVector RightTangentVec, UpTangentVec;
    GetTangentVector([&] RightTangentVec, [&] UpTangentVec);

    FVector ForceVec = RightTangentVec * XInput + UpTangentVec * YInput;
    MovementComponent->AddInputVector(ForceVec.GetSafeNormal()*MovementPower);
}
```

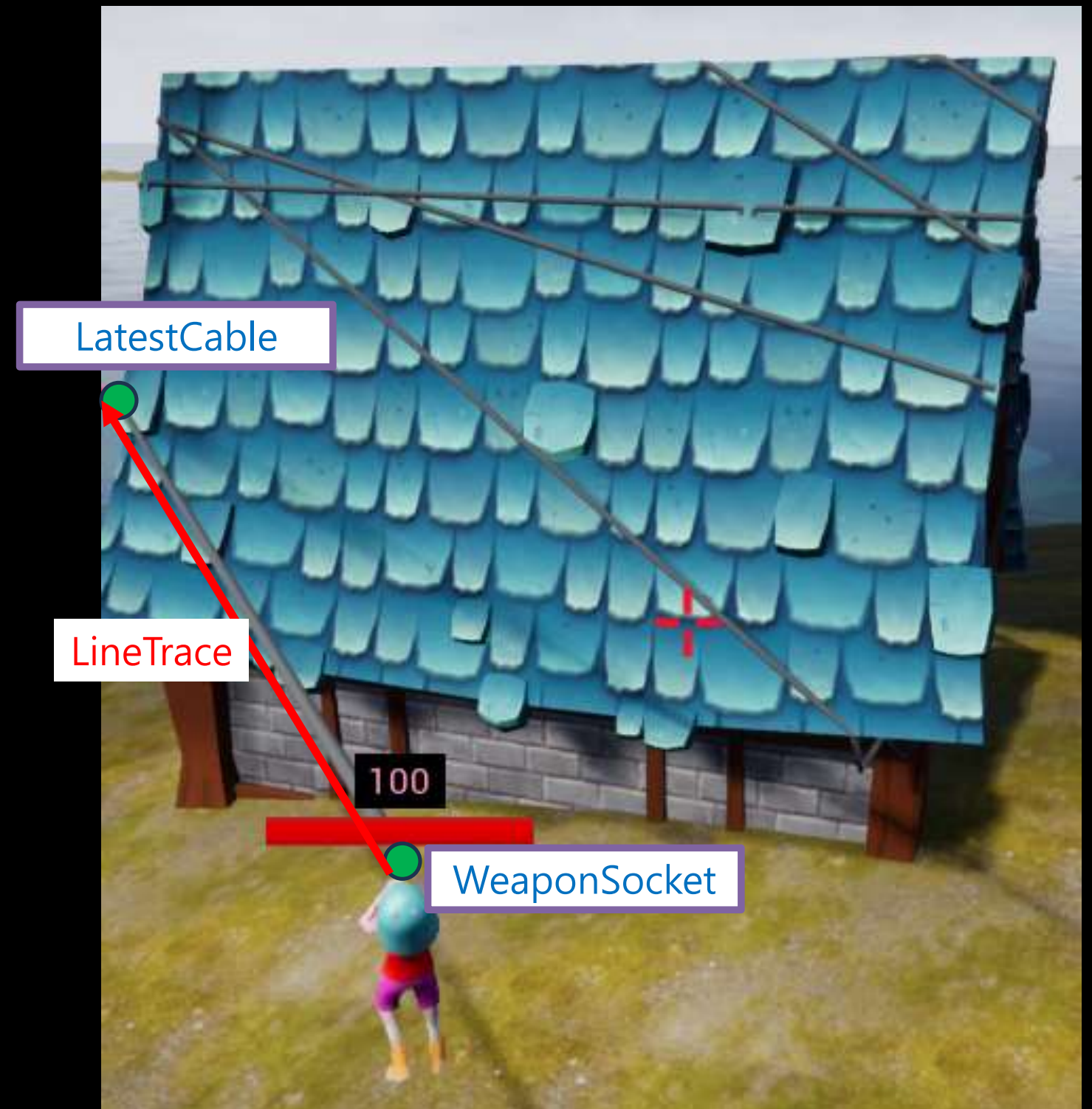
- Xinput과 Yinput은 키보드 입력으로 들어오는 값입니다.
- 현재 구의 표면에 위치하는 캐릭터의 접벡터와 Input값을 조합해 구의 표면 위를 움직이도록 이동합니다.

- 로프 감김

AMyRope

Tarray<UCableComponent*> Cables

- CableComponent를 사용해 로프를 구현했습니다.
- Cable이 어떤 물체에 걸려 감기는 것을 구현하기 위해 감기는 지점에서 캐릭터를 잇는 CableComponent를 배열에 추가합니다.
- 가장 최근에 추가된 케이블은 물체에 감긴 지점(LatestCable)과 캐릭터의 무기가 스폰되는 위치(WeaponSocket)을 연결하는 케이블입니다.
- WeaponSocket에서 LatestCable로 매 Tick마다 LineTrace를 하여 충돌하는 물체가 있는지 검사합니다.
- 충돌이 검출되면 충돌지점에서 시작하는 새로운 케이블을 생성해 WeaponSocket에 연결하고 배열에 추가합니다.



- 로프 풀림1

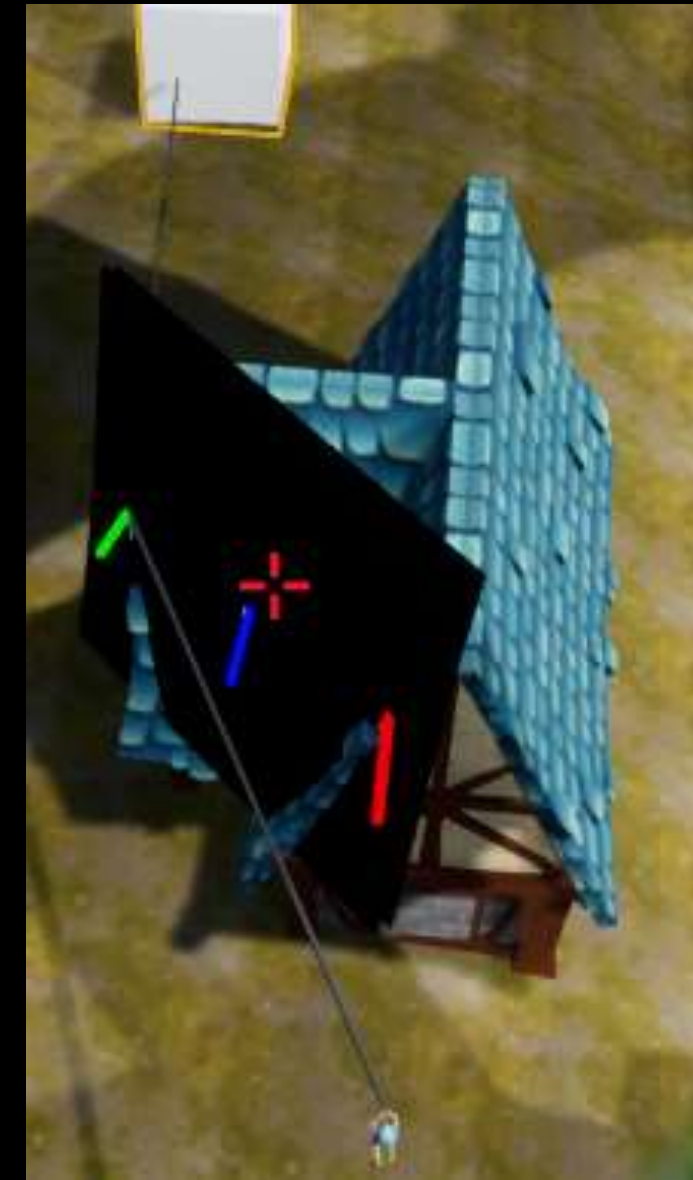
- 감겨있던 로프가 풀리는 과정을 구현했습니다.
- 우측 사진은 1번로프에서 시작해 중간에 물체에 걸려 2번, 3번 로프가 생성되었습니다.
- ② ③ 은 2번,3번 로프의 시작점(충돌지점)입니다.
- 초록, 빨강 막대는 ② ③ 위치에서의 노말벡터입니다.
파란색 막대는 2번로프의 중앙에서의 노말벡터입니다.
- 각 노말벡터를 **GreenNormal**, **RedNormal**, **BlueNormal** 이라고 할 때,
 $\text{BlueNormal} = (\text{GreenNormal} + \text{RedNormal}) * 0.5f;$
근사치로 구합니다.



- 로프 풀림2



3번로프가 평면 위쪽이 되는순간 삭제

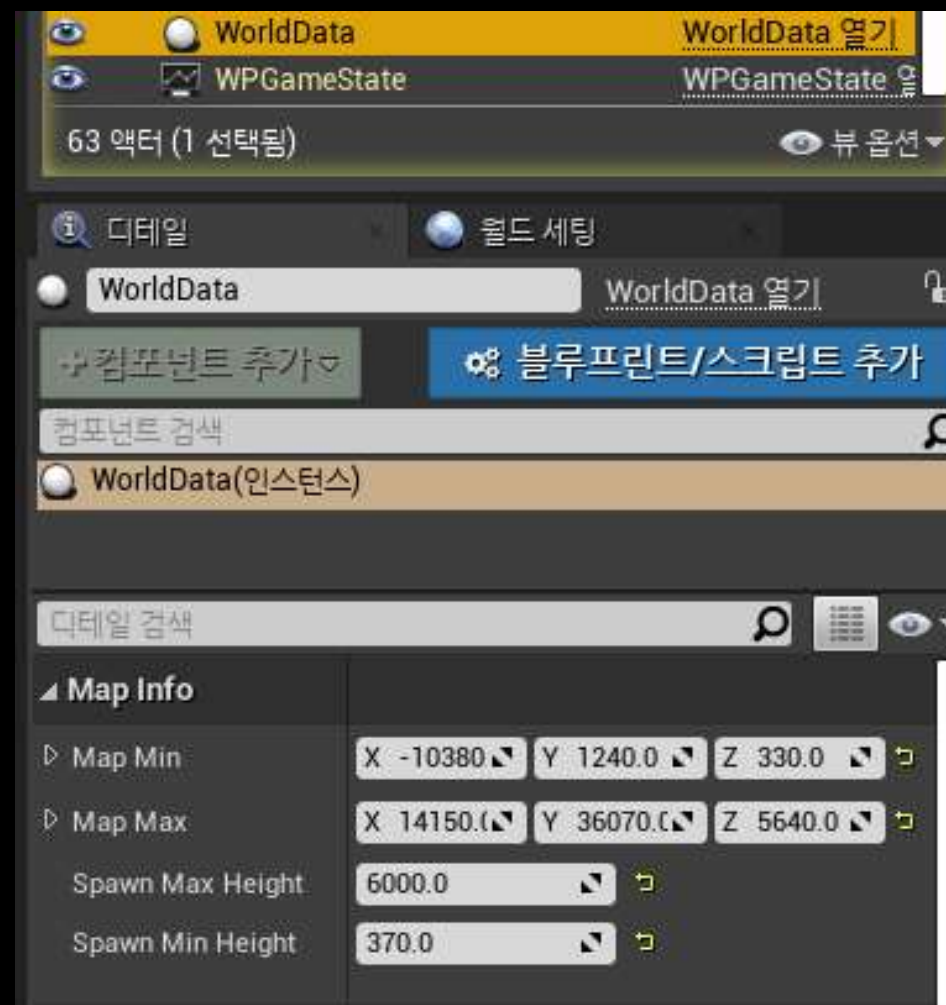


- 3번로프가 BlueNormal로 만들어진 평면의 위쪽으로 가면, 2번로프와 캐릭터 사이에 충돌하는 물체가 없는것으로 간주합니다.
- 충돌물체가 없으면 3번로프를 삭제, 2번로프의 끝을 캐릭터와 연결합니다.
- `float DotResult = DotProduct(BlueNormal, ToCableVector);`
`DotResult >= 0`인 경우 평면과 평행하거나 위쪽이라고 판단합니다.

- 캐릭터 스폰1

AWorldData

월드의 정보를 가지는 AWorldData 클래스를 만들고 레벨에 배치해 에디터에서 수정할 수 있도록 했습니다.



1. 메인메뉴에서 설정한 팀 개수와 팀원 수를 GameState에서 얻어옵니다.
총 캐릭터수 = (팀 개수 * 팀원 수)만큼 캐릭터를 스폰합니다.
2. AWorldData 클래스에서 월드의 최대,최소 좌표를 얻어옵니다.
좌표범위 내에서 X,Y값을 랜덤으로 생성하고,
생성된 X,Y값에서 스폰할 수 있는 높이가 있는지 CheckHeight 함수로 검사합니다.
결과가 True가 될때까지 랜덤좌표를 생성하고 반복합니다.

```
void AWarmsGameModeBase::SpawnAllCharacters()
{
    auto MyGameState = GetGameState<AWPGGameState>();

    //메인메뉴에서 설정한 팀 개수, 팀원 수를 얻어옵니다.
    int TeamNum = MyGameState->GetTeamNum();
    int CharacterNum = MyGameState->GetCharacterNum();

    //using Team = TArray<APlayerCharacter*>; GameState에 선언됨.
    Team Characters;
    FTransform Tr;
    float ZOffset = 100.f;

    //총 캐릭터의 수만큼 캐릭터 스폰.
    for(int i=0; i<TeamNum*CharacterNum; ++i)
    {
        bool ValidLocation = false;

        //스폰 가능한 위치를 찾을때까지 루프
        while(!ValidLocation)
        {
            FMath::RandInit(Seed: FDateTime::Now().GetTicks());

            //World 크기 내에서 랜덤한 위치를 선정합니다.
            float X, Y;
            X = FMath::FRandRange(InMin: WorldData->MapMin.X, InMax: WorldData->MapMax.X);
            Y = FMath::FRandRange(InMin: WorldData->MapMin.Y, InMax: WorldData->MapMax.Y);

            //X,Y위치에서 수직으로 LineTrace. 유효한 위치인지 검사
            FVector SpawnLocation;
            ValidLocation = CheckHeight(X, Y, [X]SpawnLocation);
            SpawnLocation.Z += ZOffset;
            Tr.SetLocation(SpawnLocation);
        }

        APlayerCharacter* SpawnedCharacter =
            (APlayerCharacter*)GetWorld()->SpawnActor(CharacterClass, &Tr);

        Characters.Add(SpawnedCharacter);
    }

    //스폰된 캐릭터 정보를 GameState에 업데이트
    MyGameState->InitTeams(Characters);

    //첫번째 팀의 랜덤한 캐릭터로 플레이어 전환
    ChangeCharacter(MyGameState->GetRandomCharacterInTeam(0));
}
```


- 캐릭터 스폰2

```
bool AWarmsGameModeBase::CheckHeight(float X, float Y, FVector& OutSpawnLocation)
{
    FHitResult OutHit;

    //up to down
    FVector Start(X,Y, InZ: WorldData->SpawnMaxHeight);
    FVector End(X, Y, InZ: WorldData->SpawnMinHeight);

    FCollisionObjectQueryParams ObjectQueryParams(ECollisionChannel::ECC_WorldStatic);

    if (GetWorld()->UWorld*
        LineTraceSingleByObjectType([&] OutHit, Start, End, ObjectQueryParams))
    {
        OutSpawnLocation = OutHit.ImpactPoint;
        UE_LOG(LogTemp, Warning, TEXT("CheckHeight Success"));

        UKismetSystemLibrary::DrawDebugLine(GetWorld(), Start, End,
            FLinearColor::Red, Duration: 10.f, Thickness: 100.f);

        return true;
    }
    else
    {
        UE_LOG(LogTemp, Warning, TEXT("CheckHeight Failed"));
    }
    return false;
}
```



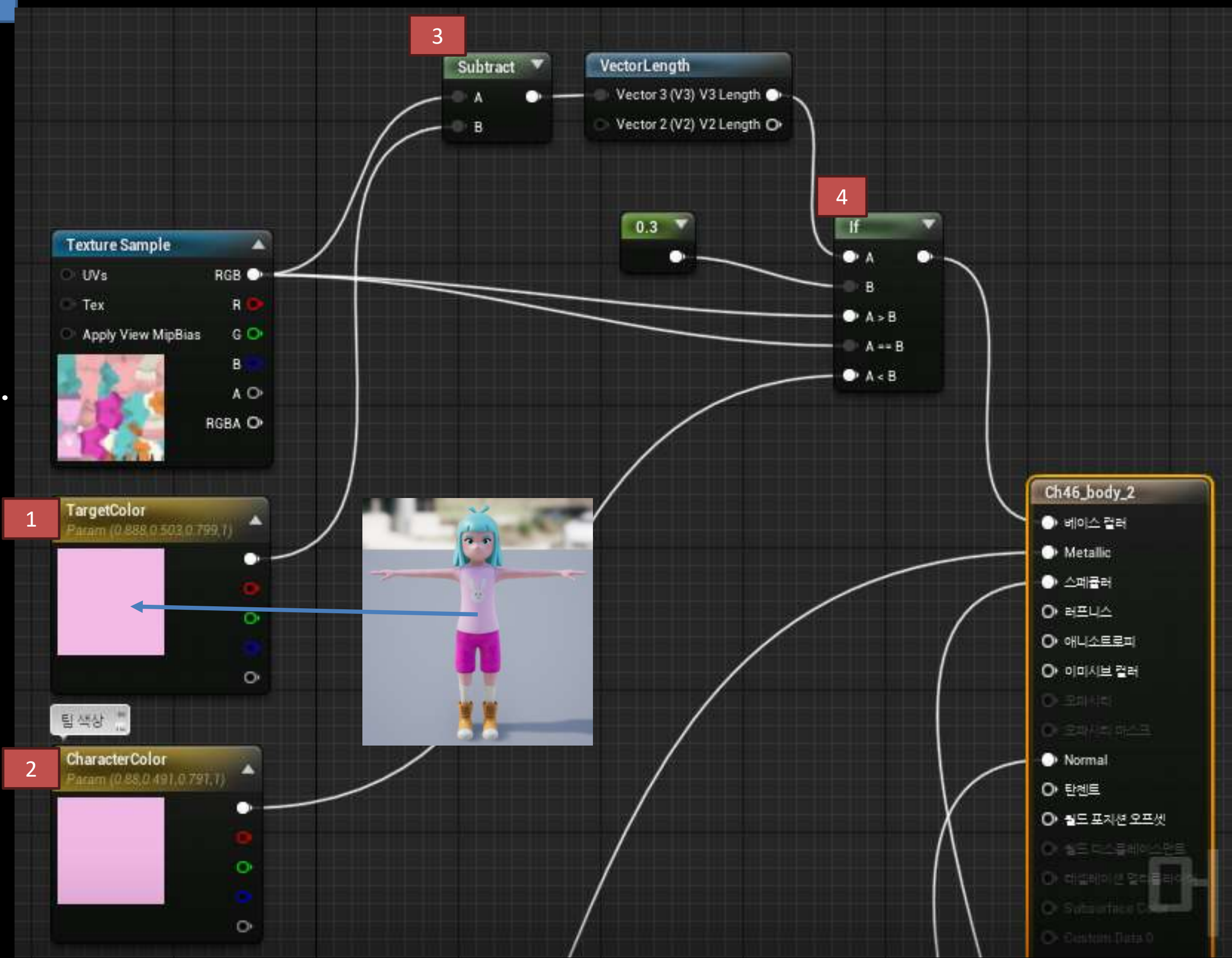
- 위 사진은 CheckHeight함수가 성공했을 때 LineTrace를 나타낸 모습입니다. 충돌지점에서 캐릭터가 스폰됩니다.

- 랜덤으로 생성한 X,Y위치에서 스폰할 수 있는 Z값을 찾습니다.
- 1. LineTrace를 위한 시작위치와 끝 위치를 초기화합니다.
Z값은 WorldData에 설정된 높이의 최대값과 최소값을 사용합니다.
- 2. 수직으로 LineTrace를 실행합니다. 캐릭터가 올라갈 수 있는 WorldStatic만 검출합니다.

- 팀 색상 설정1

- 메테리얼을 사용해 캐릭터의 상의 색상을 각 팀별로 다르게 변경했습니다.

- 1. TargetColor는 기본 텍스처의 상의 색상입니다.
- 2. CharacterColor는 상의 색상을 어떤 색으로 변경할지 결정하는 파라미터입니다. C++코드에서 설정합니다.
- 3. 픽셀값을 PixelVector라고 할 때, $\text{float PixelDiff} = (\text{PixelVector} - \text{TargetColor}).\text{Length};$
- 4. PixelDiff의 값으로 해당 픽셀이 TargetColor와 비슷한지 판단합니다. $\text{PixelDiff} < 0.3$ 일 경우 픽셀값을 CharacterColor값으로 변경합니다.



- 팀 색상 설정2

1

```
void AWPGameState::InitTeams(const Team& TeamSrc)
{
    //must TeamSrc.Num == TeamNum * CharacterNum
    check(TeamSrc.Num() == (TeamNum*CharacterNum));

    Teams.SetNum(TeamNum);

    //Add All Character.
    for(int i=0; i<TeamNum; ++i)
    {
        for(int j=0; j<CharacterNum; ++j)
        {
            Teams[i].Add(TeamSrc[CharacterNum*i + j]);
            //각 팀의 색으로 메테리얼 설정
            TeamSrc[CharacterNum*i + j]->Init([&TeamColors[i], i]);
        }
    }
}
```

2

```
void APlayerCharacter::Init(FLinearColor& Color, int TeamNum)
{
    SetColor([&] Color);
    SetTeam(TeamNum);
}

void APlayerCharacter::SetColor(FLinearColor& Color)
{
    TeamColor = Color;

    FVector ColorVec(InX: Color.R, InY: Color.G, InZ: Color.B);
    GetMesh()->SetVectorParameterValueOnMaterials(FName(TEXT("CharacterColor")),ColorVec);

    UHpBarWidget* hpBarInstance = Cast<UHpBarWidget>
        ( Src: mHpBarWidget->GetUserWidgetObject());
    hpBarInstance->SetHpBarColor(Color);
}
```

3

1. 모든 캐릭터가 스폰된 뒤, 각 팀의 정보를 초기화 하는 AWPGameState::InitTeams 함수가 호출됩니다.
2. 각 캐릭터가 i번째 팀으로 배분되고, 해당 팀으로 초기화합니다.
3. 캐릭터의 메쉬를 참조해 메테리얼에서 "CharacterColor" 파라미터를 해당 캐릭터가 속한 팀의 색상으로 설정합니다.



- 피격/사망시 카메라 전환1

APlayercharacter

FStartDamaged StartDamagedDelegate

FEndDamaged EndDamagedDeleagte

FOnDie OnDieDelegate

- 피격시작,종료,사망 3가지의
델리게이트를 정의했습니다.

1. 피격이 시작됐음을 알리는
StartDamagedDelegate.Broadcast는
캐릭터의 TakeDamage 함수에서
실행됩니다.
2. 피격이 종료됐음을 알리는
EndDamagedDelegate.Broadcast는
캐릭터의 Tick 함수에서 캐릭터가 멈추면
실행됩니다.
3. 캐릭터가 사망했음을 알리는
OnDieDelegate.Broadcast는
사망 애니메이션의 노티파이에서
실행됩니다.

```
float APlayerCharacter::TakeDamage(float Damage, FDamageEvent const& DamageEvent, AController* EventInstigator, AActor* DamageCauser)
```

```
1 StartDamagedDelegate.Broadcast(this);
```

```
void APlayerCharacter::Tick(float DeltaTime)
```

```
{  
    Super::Tick(DeltaTime);
```

```
    if (bTakingDamage)
```

```
{
```

```
        TakingDamageTime += DeltaTime;
```

```
        //데미지를 받은 이후로 캐릭터의 속도가 0이되면 턴을 종료할 준비를 함
```

```
        if (TakingDamageTime > 1.f)
```

```
{
```

```
            if (GetVelocity().IsNearlyZero(Tolerance: 5.f))
```

```
{
```

```
                UE_LOG(LogTemp, Warning, TEXT("GetVelocity : %f"), GetVelocity().Size());
```

```
                GetCapsuleComponent()->SetSimulatePhysics(false);
```

```
                SetActorRotation(FRotator::ZeroRotator);
```

```
                UE_LOG(LogTemp, Error, TEXT("Tick bTakingDamage false"));
```

```
                bTakingDamage = false;
```

```
                TakingDamageTime = 0.f;
```

```
2
```

```
                EndDamagedDelegate.Broadcast(this);
```

```
}
```

```
}
```

```
}
```

```
void UPlayerAnimation::AnimNotify_DieExplosion()
```

```
{
```

```
    APawn* Pawn = TryGetPawnOwner();
```

```
    if (!IsValid(Pawn))
```

```
        return;
```

```
    APlayerCharacter* CurrCharacter = Cast<APlayerCharacter>(Pawn);
```

```
3
```

```
    CurrCharacter->OnDieDelegate.Broadcast(CurrCharacter);
```


- 피격/사망시 카메라 전환2

AWarmsGameModeBase

```
TArray<TWeakObjectPtr<APlayerCharacter>> DamagedPlayers;  
Void SwitchDamagedCharacterCamera(float Duration, float BlendTime)  
Void SwitchMultiCamera(float Duration, float BlendTime)
```

```
void AWarmsGameModeBase::SwitchMultiCamera(float Duration, float BlendTime)  
{  
    //일정 시간마다 데미지를 받은 캐릭터 간 카메라 전환  
    1 GetWorld()->GetTimerManager().SetTimer(SwitchCameraTimerHandler,  
        FTimerDelegate::CreateLambda(  
            [&, Duration]()  
            {  
                //Switch Other Character's Camerae  
                SwitchDamagedCharacterCamera(Duration);  
                UE_LOG(LogTemp, Error, TEXT("SwitchMultiCamera In Timer"));  
            }  
        ), Duration + BlendTime, true, 0.75f);  
}
```

1. Duration : 카메라 지속시간, BlendTime : 카메라 전환시간
(Duration + BlendTime) 마다 다른 캐릭터를 탐색하도록
타이머를 실행합니다.

2. DamagedPlayerNum은 피격관련 델리게이트에 의해 변경됩니다.
피격 캐릭터가 0일 경우, SwitchMultiCamera 함수에서 실행한
타이머를 종료합니다. 사망한 캐릭터가 있을경우 해당 캐릭터로
카메라를 전환합니다.

3. DamagedPlayers는 피격중인 캐릭터의 배열이며
피격관련 델리게이트에 의해 변경됩니다.
배열에서 유효한 캐릭터를 탐색해 카메라를 전환합니다.

```
{  
    static int CurrentIdx = -1;  
    UE_LOG(LogTemp, Error, TEXT("SwitchDamagedCharacterCamera Duration : %f"), Duration);  
    2 if (DamagedPlayerNum == 0)  
    {  
        GetWorld()->GetTimerManager().ClearTimer([*] SwitchCameraTimerHandler);  
        UE_LOG(LogTemp, Error, TEXT("SwitchDamagedCharacterCamera ClearTimer"));  
        CurrentIdx = -1;  
        //이번턴에 죽은 플레이어가 있을 때 죽는장면 화면전환  
        if(DeadPlayers.Num() != 0)  
            SwitchDeadCharacterCamera(Duration, BlendTime);  
        else  
            NextTurn();  
        return;  
    }  
    3 for(int i=0; i<DamagedPlayers.Num(); ++i)  
    {  
        UE_LOG(LogTemp, Error, TEXT("SwitchDamagedCharacterCamera for"));  
        auto Character : TWeakObjectPtr<APlayerCharacter> = DamagedPlayers[i];  
        if (!CheckDamagedPlayer(Character))  
        {  
            RemoveDamagedPlayer(Character);  
        }  
        else  
        {  
            CurrentIdx = i;  
            break;  
        }  
    }  
    if (CurrentIdx == -1)  
        return;  
    DamagedPlayers[CurrentIdx].Get()->ActiveOneCamera(1);  
    GetWorld()->GetFirstPlayerController()->APlayerController*  
        SetViewTargetWithBlend(DamagedPlayers[CurrentIdx].Get(), BlendTime);  
}
```

**마지막 페이지 입니다.
감사합니다!**
